

Temperature Predictor: An End-to-End IoT and Machine-Learning System for Seven-Day Temperature Forecasts

Constantine Giantselidis

[TODO: Department]

[TODO: University]

Thessaloniki, Greece

[TODO: e-mail]

Abstract—Weather forecasts describe grid cells and airport stations, not the balcony where a sensor hangs. This paper presents Temperature Predictor, an end-to-end system that turns a low-cost sensor into a local seven-day forecast of daily maximum and minimum temperature. An ESP8266 microcontroller with a DHT22 sensor posts a reading every minute to a FastAPI service, which authenticates the device and stores the reading in PostgreSQL. A nightly worker aggregates the readings into daily statistics, runs fourteen ridge regression models, one per target and forecast day, and stores the forecasts, which a Streamlit dashboard reads through the API. Because a new sensor has no history, the models are trained on ERA5-Land reanalysis data for Thessaloniki, restricted to the six daily variables the sensor itself measures. On 616 held-out days, the maximum-temperature model reaches a mean absolute error of 1.62 °C for the next day and 2.74 °C at seven days.

Index Terms—Internet of Things, temperature forecasting, ridge regression, ERA5-Land, REST API, machine learning systems

I. INTRODUCTION

Weather services forecast for grid cells and stations, while a sensor on a specific balcony measures conditions that neither describes exactly. Low-cost microcontrollers make such measurements cheap. This project asks a practical engineering question: can one low-cost sensor feed a complete, maintainable forecasting system, from the hardware to a dashboard?

Temperature Predictor answers with a working pipeline. A sensor posts readings; an API authenticates and stores them; a worker turns them into daily statistics and seven-day forecasts of maximum and minimum temperature; a dashboard shows both. A first version (2024) used XAMPP, PHP, a local MySQL database and a ridge model trained on station data. When its database was lost, the system was rebuilt in 2026 with production practices: a containerised database, a typed, self-documenting API, device authentication, and a decision log that records every design choice with its rationale and rejected alternatives, in the style of architecture decision records [1].

The rebuild had a fixed deadline and was planned risk-first. The hardware, the only part that could fail for reasons outside the code, was built first under a timebox, with a software

simulator as the fallback. Work that did not serve the end-to-end path was moved to future work.

Section II reviews the background, Section III the architecture, Section IV the data and model, and Section V the results. Section VI discusses limitations and Section VII concludes.

II. BACKGROUND

Operational forecasts come from numerical weather prediction, which simulates atmospheric physics forward from an estimate of the current state. Data-driven models have recently become competitive: WeatherBench [2] set a common benchmark, and GraphCast [3], a graph neural network trained on decades of reanalysis, outperformed a leading operational system on most targets up to ten days ahead. Such models are global and use hundreds of variables. This project sits at the opposite end: one location, six variables and a linear model.

Forecasts are judged against simple reference forecasts [4], [5]. Persistence predicts that a future day will equal today; climatology predicts the long-term average for the date. A model that cannot beat them adds nothing.

A reanalysis combines a weather model with historical observations to produce a consistent, gap-free record of past weather. ERA5 [6] covers the globe hourly at about 31 km resolution, and ERA5-Land [7] reruns its land component at about 9 km. Both are available through the Open-Meteo API [8].

Ridge regression [9] fits a linear model by least squares with an L2 penalty that shrinks the coefficients. The penalty keeps the fit stable when inputs are strongly correlated, as a day's maximum, minimum and mean temperature are. The models were built with scikit-learn [10].

III. SYSTEM ARCHITECTURE

A. Overview

Fig. 1 shows the data flow. Every 60s the device posts a JSON reading (device ID, temperature, humidity) to `POST /readings`, with its token in the `Authorization` header. The API validates and authenticates the request and inserts a row into the `dht22` table. Every night a worker condenses each device's readings into one row per day in

Insert architecture diagram: ESP8266 + DHT22 → FastAPI
 → PostgreSQL ← worker (aggregator, predictor, scheduler);
 Streamlit dashboard → FastAPI

Fig. 1. System architecture. The sensor posts readings to the API, the worker turns them into daily aggregates and stored forecasts, and the dashboard reads everything through the API.

`dht22_aggregate` (maximum, minimum and mean temperature and humidity, and a reading count), then computes seven-day forecasts and stores them in a `prediction` table. The dashboard reads devices, readings and forecasts through GET endpoints, never from the database directly, so database credentials live in one service only.

B. Technology Stack

The 2024 stack was replaced piece by piece: XAMPP by Docker Compose, so the database is recreated with one command; PHP by FastAPI, which validates requests with Pydantic and publishes an OpenAPI description of every endpoint; MySQL by PostgreSQL with PostGIS, for strict types, CHECK constraints, time-zone-aware timestamps and geographic locations. With a Streamlit dashboard, the whole system is written in Python.

C. API and Data Integrity

Database models (SQLAlchemy) and API schemas (Pydantic) are separate classes, so the API can hide columns such as the token hash and accept inputs that are not columns. Physical ranges are enforced twice, as defence in depth: the schema rejects a temperature outside -80 to 80°C or a humidity outside 0–100 % with a clear 422 error, and database CHECK constraints reject the same values from any other writer. Because the ESP8266 has no reliable clock, the server stamps readings that arrive without a time. Devices are created only through POST `/devices`, never by manual SQL, so hashing and defaults always apply. GET `/readings` accepts an optional device filter and a bounded limit and returns the newest rows first.

D. Security

A device authenticates with a random token in the standard `Authorization: Bearer` header, which works for every HTTP method and keeps the secret out of URLs and logs. The server generates the device ID (a random UUID, which cannot be guessed or chosen) and the token, and returns the raw token only once, at registration, as cloud providers do with API keys. Only a SHA-256 hash is stored, so a leaked database holds no usable credentials; a fast hash suffices because the tokens are long random strings, not human passwords. Secrets

never enter version control: the WiFi and token values live in a git-ignored header on the device, and the repository’s history was rewritten before publication to remove private notes and e-mail addresses. One gap remains: registration itself is unauthenticated, which is acceptable on a home network but not for a public deployment.

E. Worker: Aggregation, Prediction and Scheduling

Batch work runs in a separate worker process, never inside the API: the API must answer sensors quickly, and a failing batch job must not stop ingestion, nor the reverse. Aggregation is one `INSERT ... SELECT ... GROUP BY` statement executed inside PostgreSQL, so raw readings never leave the database, and its `ON CONFLICT DO UPDATE` clause turns it into an upsert that can be re-run without duplicating rows. The SQL lives in the worker’s Python files, not in stored procedures, so it is versioned with the code that uses it, while the database keeps the job of storing and guarding the data. The predictor’s input query, which selects each device’s newest complete day, follows the same rule. APScheduler runs aggregation and then prediction every night at 00:10 Athens time; a failed run is logged without stopping the scheduler, and because both steps are upserts, retries are safe.

F. Forecast Storage and Serving

Forecasts are computed once a day and stored, not computed per request: their inputs change only daily, stored forecasts build a history that can later be compared with what the sensor measured, and the API needs no machine-learning libraries. The `prediction` table has one row per device, base date and target date; that triple is both the primary key and the upsert key, and CHECK constraints require the target date to follow the base date and the temperatures to be plausible. GET `/prediction` returns a device’s newest forecast, or an older one for a given base date, sorted by target date.

G. Firmware

The firmware adapts the 2024 sketch: it sends JSON with the Bearer header and reads its credentials from the git-ignored header. Because the sensor runs on USB power on a balcony, the board stays awake and waits 60 s between readings (about 1,440 per day); deep sleep, which needs an extra wake-up wire and reconnects WiFi on every cycle, is kept for a future battery version. The hardest problem was the network: on a WiFi 6 router, the ESP8266, a 2014 design limited to 802.11b/g/n, joined the network but never received a DHCP address, fell back to a link-local 169.254.x.x address, and every request failed. A static address in the firmware solved it.

IV. DATA AND MODEL

A. Training Data

A new sensor has no history, so the model learns from decades of city weather and is then applied to sensor data. Version 1 used NOAA GHCN-Daily records from the Thessaloniki airport station [11]. Inspection showed that its targets were its weakest columns: 32.5 % of daily maxima and 41.4 %

of daily minima were missing (21,211 rows, 1964–2025), the gaps grew over time (355 of 365 maxima missing in 2014), and two years were absent. Version 1 had forward-filled the gaps, turning about 40% of its targets into copies of earlier days, which probably flattered its reported next-day error of about 1 °C. The rebuild therefore trains on ERA5-Land [7], [12] for the city centre, downloaded through Open-Meteo: 28,017 gap-free daily values from 1 January 1950 to 15 September 2026. The price is that a reanalysis describes a 9 km grid cell, not a thermometer, and smooths extremes slightly. Cleaning dropped one incomplete day and renamed the columns to exactly the names of the sensor’s aggregate table, so the predictor needs no translation step. The raw file is committed to the repository; derived files are rebuilt by code. Training starts in 1979, when satellite observations enter the reanalysis, which also leaves out the colder climate of earlier decades.

B. Features and Targets

Every input must be available from the sensor at prediction time; otherwise the model could be trained but never served, a case of training–serving skew [13]. The inputs are therefore the six daily variables of the aggregate table: maximum, minimum and mean temperature and relative humidity. The targets are the maximum and minimum temperature of each of the next seven days. Each target is the series shifted back by n days, so every row pairs one day’s inputs with the values observed n days later; the last seven days, which have no complete future, are dropped, and no input ever looks forward in time. Humidity is only an input: the DHT22 measures it less reliably, and it depends on rain and wind, which the model does not see.

C. Models and Evaluation

A linear model was chosen on purpose: with a few correlated tabular inputs, better features help more than a more complex model, and baselines keep the result honest. Fourteen independent ridge models ($\alpha = 0.1$), one per target and forecast day, are saved with their feature lists in one `joblib` file that the worker loads. The v1 recipe used today’s maximum and minimum, a 30-day rolling mean of the maximum and today’s deviation from it, which creates a cold start: a new sensor would need 30 complete days before its first forecast. Version 2 uses only the six same-day variables, so one complete day suffices. The data are split by time, never shuffled [5]: training on 1979–2024 and testing on 616 days, from 1 January 2025 to 8 September 2026. Accuracy is measured with the mean absolute error

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|, \quad (1)$$

in degrees Celsius, where y_i is the observed and $\hat{y}_i$ the predicted value; it is easy to interpret and, unlike the root mean square error, not dominated by rare large misses. [TODO: The minimum-temperature models were still trained on 1950–2021 and tested from 2022; retrain them on the same split and update this sentence.] To meet the deadline, time-series cross-validation, automatic feature selection and tuning of α were

TABLE I
MEAN ABSOLUTE ERROR (°C) ON THE TEST PERIOD. PERSIST.: PERSISTENCE BASELINE (DAY n EQUALS TODAY)

Day	Maximum		Minimum	
	Persist.	Ridge v2	Persist.	Ridge v2
1	??	1.62	??	1.10
2	??	2.19	??	??
3	??	2.44	??	??
4	??	2.54	??	??
5	??	2.66	??	??
6	??	2.73	??	??
7	??	2.74	??	??

Insert dashboard screenshot (readings chart + 7-day forecast)

Fig. 2. Streamlit dashboard: recent readings of the balcony sensor and its seven-day forecast.

deferred, and training still lives in a notebook rather than a script.

V. RESULTS

Table I compares the version-2 models with the persistence baseline on the test period. Next-day maximum temperature is predicted with an MAE of 1.62 °C. The error grows with the horizon but flattens after day 4, reaching 2.74 °C on day 7: the further ahead the target, the less today’s weather tells the model, and its predictions drift toward the typical value for the season. The v1 recipe scores the same 1.62 °C on day 1 but 2.91 °C on day 7, so dropping the 30-day features removed the cold start without losing accuracy. The next-day minimum is predicted with an MAE of 1.10 °C. [TODO: One sentence on persistence once Table I is complete: by how much does the model beat it, and from which day on?]

On gap-free data, the v1 recipe’s next-day error is 1.62 °C, against the roughly 1 °C that v1 reported on forward-filled station data. Sources and periods differ, so this is no controlled comparison, but it is consistent with the copies having flattered v1.

The whole chain works end to end: the balcony sensor stored its first reading on 23 September 2026 (26.5 °C, 33.6 % relative humidity), and readings, daily aggregates and forecasts now flow from the sensor to the dashboard (Fig. 2).

VI. LIMITATIONS AND LESSONS

Garbage in, garbage out. The balcony sensor’s first “day” held only 23 readings, about 23 minutes, with maximum and minimum almost equal. No training day has such a small

range, and the model extrapolated to maxima of about 34 °C. The predictor should skip days with too few readings.

Domain shift. The model learns from a 9 km reanalysis cell but is applied to a balcony, where sun and walls bias readings; the DHT22 is specified to $\pm 0.5^{\circ}\text{C}$ [14] and is weaker on humidity. Once weeks of data exist, overlapping days can be used to estimate and correct the offset.

Day boundary. Training days are Athens calendar days, but the aggregator groups readings by UTC date. Since the nightly job runs at 00:10 Athens time, still the previous day in UTC, the newest forecast uses data one day older than intended.

Model scope. Version 2 has no time-of-year feature and no information about approaching weather systems, which limits its skill beyond the first days; physics-based forecasts will be far better there.

Evaluation. The results come from one time split, the persistence comparison is pending, and the minimum models use a different split.

The broader lesson echoes production ML experience [15]: the model was the easy part. A charge-only USB cable, a DHCP failure on a WiFi 6 router, missing training values and the cold start each cost more effort than fitting the regression.

VII. FUTURE WORK AND CONCLUSION

Future work falls into three groups. Model: a minimum-reading check, day-of-year features, cross-validation with feature selection and tuning, a training script, calibration against the sensor, and a real-world evaluation that joins stored forecasts with the aggregates later measured for the same dates. Operations: containers with restart policies for the API, worker, scheduler and dashboard, TLS behind a reverse proxy, an administrator key for registration, duplicate-safe ingestion and geographic device locations. Hardware: a battery version with deep sleep, and a router address reservation instead of a static address.

Temperature Predictor shows that one low-cost sensor can feed a complete, maintainable forecasting system. The decisive steps concerned data, not algorithms: replacing an incomplete station record with a gap-free reanalysis, and restricting the model to what the sensor can provide, which also removed its cold start.

ACKNOWLEDGMENT

[TODO: Check your course's rules on AI use and edit this statement.] Claude (Anthropic), an AI assistant, was used as a mentor during development: it explained concepts, reviewed the code the author wrote and supplied short fixes. It also produced a first draft of this report from the project's decision log, which the author then revised.

REFERENCES

- [1] M. Nygard, "Documenting architecture decisions," Cognitect Blog, Nov. 15, 2011. [Online]. Available: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- [2] S. Rasp, P. D. Dueben, S. Scher, J. A. Weyn, S. Mouatadid, and N. Thuerey, "WeatherBench: A benchmark data set for data-driven weather forecasting," *J. Adv. Model. Earth Syst.*, vol. 12, no. 11, Art. no. e2020MS002203, 2020, doi: 10.1029/2020MS002203.
- [3] R. Lam *et al.*, "Learning skillful medium-range global weather forecasting," *Science*, vol. 382, no. 6677, pp. 1416–1421, 2023, doi: 10.1126/science.adi2336.
- [4] D. S. Wilks, *Statistical Methods in the Atmospheric Sciences*, 4th ed. Amsterdam, The Netherlands: Elsevier, 2019.
- [5] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>
- [6] H. Hersbach *et al.*, "The ERA5 global reanalysis," *Q. J. R. Meteorol. Soc.*, vol. 146, no. 730, pp. 1999–2049, 2020, doi: 10.1002/qj.3803.
- [7] J. Muñoz-Sabater *et al.*, "ERA5-Land: A state-of-the-art global reanalysis dataset for land applications," *Earth Syst. Sci. Data*, vol. 13, no. 9, pp. 4349–4383, 2021, doi: 10.5194/essd-13-4349-2021.
- [8] P. Zippenfenig, "Open-Meteo.com Weather API," Zenodo, 2023, doi: 10.5281/zenodo.7970649. Data licensed under CC BY 4.0.
- [9] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.
- [10] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [11] M. J. Menne, I. Durre, R. S. Vose, B. E. Gleason, and T. G. Houston, "An overview of the Global Historical Climatology Network-Daily database," *J. Atmos. Ocean. Technol.*, vol. 29, no. 7, pp. 897–910, 2012, doi: 10.1175/JTECH-D-11-00103.1.
- [12] J. Muñoz Sabater, "ERA5-Land hourly data from 1950 to present," Copernicus Climate Change Service (C3S) Climate Data Store (CDS), 2019, doi: 10.24381/cds.e2161bac.
- [13] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ML test score: A rubric for ML production readiness and technical debt reduction," in *Proc. IEEE Int. Conf. Big Data*, 2017, pp. 1123–1132.
- [14] Aosong Electronics, "Digital-output relative humidity & temperature sensor/module DHT22 (DHT22 also named as AM2302)," datasheet.
- [15] D. Sculley *et al.*, "Hidden technical debt in machine learning systems," in *Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 2503–2511.